

Guion de presentación: KnowGraph

Defensa de proyecto

Introducción

Diapositiva: Título

(Sin hablar, se muestra la portada mientras el tribunal se acomoda)

Diapositiva: Contenidos

(Iniciar tras pasar a esta diapositiva)

Buenos días/tardes. Mi nombre es Valentino Russo y hoy presento mi proyecto final de ingeniería, el título de la aplicación es **KnowGraph**.

En esta exposición recorreré primero la motivación y el contexto, los conceptos necesarios para entender la herramienta, una demostración de su funcionamiento, los detalles técnicos para los desarrolladores y, finalmente, una reflexión sobre sus puntos fuertes y débiles, junto con posibles mejoras. La idea es que tanto quien viene del mundo técnico como quien no, pueda seguir el hilo sin dificultad.

Sección 1: Motivación y proyectos similares

Diapositiva: Motivación

El punto de partida es una necesidad real: explorar grandes volúmenes de datos interconectados sin ser un experto en grafos de conocimiento ni en SPARQL. Queríamos una herramienta que permitiera consultar, visualizar y transformar datos con una curva de aprendizaje baja, pero sin sacrificar potencia. KnowGraph integra ejecución de consultas, visualización interactiva (tanto en grafo como mediante reducción dimensional) y carga de datos tabulares, todo en un solo entorno.

Diapositiva: Proyectos y herramientas relevantes

No partimos de cero. Existen referentes como Protégé, centrado en ontologías; Stardog, una base de datos semántica empresarial; y servicios como Wikidata Query Creator o DBpedia, que ofrecen consultas visuales sobre grandes grafos públicos. Sin embargo, estas herramientas suelen estar orientadas a un perfil muy técnico o ligadas a un único proveedor. KnowGraph busca ser más accesible, combinando las ventajas de un entorno local con la capacidad de conectarse a servicios externos.

Sección 2: Conocimientos necesarios

Diapositiva: Conocimientos necesarios para utilizar la aplicación

Aunque la aplicación intenta ocultar la complejidad, hay cuatro pilares conceptuales que conviene tener presentes: qué es un grafo, cómo se modela conocimiento con RDF, cómo se interroga ese conocimiento con SPARQL y cómo Docker facilita el despliegue. Los revisaremos brevemente.

Diapositiva: Grafos y representación de conocimiento

Un grafo es simplemente un conjunto de nodos (entidades) y aristas (relaciones). Dependiendo de si la relación tiene dirección, el grafo puede ser dirigido o no; además, se pueden añadir etiquetas que describen el significado de cada relación. Esta estructura es muy natural para modelar cualquier dominio: personas, ciudades, genes, transacciones...

Diapositiva: Grafos de conocimiento

Cuando a ese grafo le añadimos un modelo semántico explícito (es decir, definimos qué significan realmente las relaciones y las entidades), hablamos de grafos de conocimiento. Son la columna vertebral de la Web Semántica y del Linked Data, y permiten integrar información de fuentes heterogéneas razonando sobre ella.

Diapositiva: RDF y su conexión con los grafos

El estándar técnico que materializa esto es RDF, Resource Description Framework. Define un modelo basado en triplas: sujeto, predicado, objeto. Cada tripla es una arista dirigida y etiquetada. RDF es la base sobre la que luego se construyen las consultas. Es interoperable y procesable automáticamente.

Diapositiva: SPARQL

Para extraer información de esos grafos RDF utilizamos SPARQL, el lenguaje de consulta estándar del W3C. Frente a SQL, SPARQL permite navegar las relaciones del grafo de forma mucho más expresiva sin necesidad de múltiples JOINS. Por eso fue la elección natural para este proyecto.

Diapositiva: Docker

Por último, Docker. Es la herramienta que nos permite empaquetar todos los servicios (backend Java, servicios Python, frontend) en contenedores ligeros. El usuario solo necesita instalar Docker y ejecutar un comando. La primera instalación descarga las imágenes, por lo que tarda unos minutos, pero las siguientes son casi inmediatas.

Sección 3: Comportamiento de la aplicación y manual de usuario

Diapositiva: Comportamiento de la aplicación y manual de usuario

Veamos ahora cómo se usa KnowGraph. *(Señalar los puntos de la diapositiva)*

Diapositiva: Ejecución de consultas

El flujo más básico es el de ejecutar una consulta SPARQL. El usuario escribe la consulta o la genera visualmente, pulsa ejecutar y obtiene una tabla paginada con los resultados, que puede descargar en CSV o JSON. Además, hay un chat con un asistente basado en inteligencia artificial para resolver dudas sobre SPARQL. *(Señalar la captura de pantalla)*

Diapositiva: Generador visual de consultas SPARQL

Para quienes no conocen SPARQL, disponemos de un “Query Wizard”. Se organiza en tres bloques: prefijos (para acortar URIs), datasets (opcional) y la construcción paso a paso de la consulta, donde se seleccionan variables, filtros, límites... Al finalizar, la consulta se copia automáticamente en el editor.

Diapositiva: Constructor de triplas RDF para inserciones

De forma análoga, hay un asistente para construir inserciones de datos. Se definen triplas (sujeto, predicado, objeto) con ayuda de prefijos, se elige el grafo destino y se genera automáticamente la consulta INSERT. Pensado para usuarios que quieran poblar sus propios grafos sin escribir SPARQL manualmente.

Diapositiva: Cambio de proveedor de servicios SPARQL

Una característica importante es la posibilidad de cambiar el motor que responde las consultas. Por defecto, usamos el servidor SPARQL local que incluye el proyecto, y eso nos da control total: cargar datos propios, exportarlos a cualquier formato, etc. Pero también podemos conectarnos a servicios públicos como Wikidata para lanzar consultas sobre sus grafos, aunque en ese caso se pierden las capacidades de carga y exportación local. Es un equilibrio entre autonomía y alcance.

Diapositiva: Visualización de datos en KnowGraph

Tras obtener resultados, podemos visualizarlos de dos maneras.

- La primera es como un **grafo interactivo**, en 2D o 3D, donde el usuario decide qué variables serán nodos y cuáles arcos.
- La segunda es mediante **reducción dimensional**: algoritmos como PCA, t-SNE o UMAP proyectan los datos multidimensionales en dos o tres dimensiones, permitiendo detectar agrupaciones o patrones ocultos.

Todo esto se procesa bajo demanda y se cachea para no repetir cálculos costosos.

Diapositiva: Ejemplo de uso en la aplicación

(Podría mostrarse una breve demo grabada o describir un flujo concreto). Por ejemplo, cargar un CSV de películas, convertirlo a triplas RDF, ejecutar una consulta que relacione directores y géneros, y luego visualizar el resultado en un grafo 3D.

Sección 4: Documentación para desarrolladores

Diapositiva: Documentación para desarrolladores

Para los compañeros técnicos, entramos en la arquitectura del sistema.

Diapositiva: Introducción y arquitectura general

KnowGraph sigue una arquitectura de microservicios. Tenemos un backend Java (que maneja el núcleo SPARQL con Apache Jena y Spring Boot), dos servicios Python (importer y visualizer) y un frontend React servido con Nginx. Todo está contenerizado con Docker y orquestado con Docker Compose. La comunicación se canaliza a través de Nginx, que actúa como proxy inverso.

Diapositiva: Backend Java

El servicio Java utiliza Apache Jena para todas las operaciones sobre grafos RDF: carga, consulta, razonamiento... Spring Boot expone una API REST y Maven se encarga de construir el artefacto final.

Diapositiva: Backend Python

Los dos servicios Python cumplen roles específicos.

- El **Importer** es un Flask que recibe archivos (CSV, JSON, Excel, XML), los convierte en triplas RDF y las envía al backend Java.
- El **Visualizer**, también Flask, toma datos tabulares, aplica reducción de dimensionalidad con scikit-learn y UMAP, y devuelve coordenadas listas para graficar.

Ambos utilizan Gunicorn como servidor de producción.

Diapositiva: Frontend

El frontend está construido con React y TypeScript, usando Vite como bundler. Las rutas se gestionan con React Router y las peticiones a la API se redirigen mediante el proxy de Nginx. Proporciona la interfaz para todos los módulos: ejecución, construcción de consultas, carga de datos y visualización.

Diapositiva: Integración y flujos de trabajo

El flujo típico es: el usuario interactúa con el frontend; este envía solicitudes HTTP a Nginx en el puerto 80; Nginx enruta según la ruta al servicio correspondiente (sparql-backend, importer, visualizer). Docker Compose asegura que todos los contenedores estén en la misma red y se levanten en orden.

Diapositiva: Docker: contenerización y orquestación

Cada servicio se define en el archivo `docker-compose.yaml`: imágenes, redes, volúmenes (como el volumen para los datos del Importer), healthchecks... Esto garantiza portabilidad total.

Diapositiva: Nginx como proxy inverso

Nginx es la puerta de entrada. Unifica el acceso, sirve los estáticos y evita problemas de CORS. Además, hemos configurado timeouts amplios porque algunas operaciones de reducción dimensional pueden tardar varios segundos.

Diapositiva: Puntos fuertes, débiles y posibles mejoras

(Aquí se disparan las tres diapositivas siguientes)

Diapositiva: Puntos fuertes

Entre los puntos fuertes destacan:

- Despliegue sencillo con Docker.
- Potencia sobre datos locales (importación, transformación, consulta y exportación).
- Doble modalidad de visualización (grafo y reducción dimensional).
- Asistente basado en LLM que reduce la barrera de entrada a SPARQL.

Diapositiva: Puntos débiles

Reconozcamos las limitaciones:

- La dependencia de Docker puede ser una traba para usuarios no técnicos; la primera ejecución descarga muchas imágenes.
- La imagen del Visualizer no es la más ligera (usamos `python:slim` en vez de `alpine`).
- Al cambiar a un proveedor externo se pierden funcionalidades importantes.
- A nivel de código, hay deuda técnica y falta de patrones de diseño claros.

Diapositiva: Posibles mejoras

En el futuro proponemos:

- Una refactorización completa para mejorar la mantenibilidad.
- Ampliar el Importer a más formatos y fuentes remotas.
- Incorporar más algoritmos de visualización.
- Añadir una guía de ayuda interactiva.
- Dotar a la tabla de resultados de una barra de búsqueda y filtrado.

Todo esto haría la herramienta aún más robusta y accesible.

Bibliografía

Diapositiva: Referencias bibliográficas

Gran parte del trabajo se sustenta en el libro *Learning SPARQL* de Bob DuCharme y en el artículo *Knowledge Graphs* de Hogan et al. (2020). Además, se ha hecho un uso intensivo de la documentación oficial de Apache Jena, Spring Boot, Docker, Vite y Nginx, así como de los fundamentos de Linked Data propuestos por Tim Berners-Lee. Todo este material queda a su disposición para cualquier consulta.

Cierre

Diapositiva: Cierre

Y hasta aquí la presentación. Muchas gracias por su atención. Quedo a su disposición para cualquier pregunta.